Formal Analysis of Reaction Systems Using Maude Strategies

Demis Ballis[1], Linda Brodo[2], Moreno Falaschi[3], and Carlos Olarte[4]

[1] DMIF, University of Udine, Italy
[2] Dipartimento di Scienze economiche e aziendali, Università di Sassari, Italy
brodo@uniss.it.
[3] Dipartimento di Ingegneria dell'Informazione e Scienze Matematiche
Università di Siena, Italy moreno.falaschi@unisi.it.
[4] LIPN, CNRS UMR 7030, Université Sorbonne Paris Nord, France
olarte@lipn.univ-paris13.fr

Abstract. Reaction Systems (RSs) are computational models inspired by biological systems, where entities are produced by reactions and may enable or inhibit other reactions. The interaction of an RS with its environment is modeled through sets of entities, called contexts. We introduce an executable semantics for RSs formalized as a fully nondeterministic rewrite theory. By generating contexts from all possible subsets of external stimuli at each step, the model explores all potential environmental interactions without predetermining a specific context sequence. To mitigate the state explosion induced by unrestricted nondeterminism, our execution model relies on rewriting strategies implemented in the Maude rewriting engine. These strategies make it possible to prune the computation tree and identify relevant execution paths. Our executable specification supports a wide range of verification tasks, including reachability analysis via strategic rewriting, LTL/CTL model checking, context-sequence synthesis for dynamically deriving environmental inputs, and conditional context generation via perfect-recall strategies. We apply our formal methodology to a breast cancer case study. Our analyses not only confirm existing results from the literature, but also support the validation of new hypotheses.

Keywords: Reaction Systems · Rewriting Logic · Model Checking.

1 Introduction

Reaction Systems (RSs) [6] are a bio-inspired computational framework applicable to both biological modeling [2, 4, 8] and theoretical computing foundations [10, 11]. A RS is built from a finite set of entities and a finite set of reactions acting on entities. A reaction is a triple (R, I, P) where R is the set of *reactants* (entities whose presence is needed to enable the reaction), I is the set of *inhibitors* (entities whose absence is needed to enable the reaction) and P is the set of *products* (entities that are produced if the reaction is enabled). The behavior of a RS is typically specified as a rewrite system that models *processes*

interacting with an external *context* that supplies environment entities at each step. The current system state is determined by the union of the entities coming from the environment with those produced in the previous state. A state transition is then determined by applying all and only the enabled reactions to the entitites in the current state.

Rewriting Logic (RL) [14] is a flexible semantic framework for modeling complex systems, and is efficiently implemented in the high-performance rewriting engine Maude [7]. The basic specification unit in RL is a *rewrite theory*, which combines an equational theory, used to represent system states as terms of an algebraic datatype, with a set of rewrite rules that model transitions between system states. Maude's default engine executes any applicable rule at each step, which may lead to state explosion in highly nondeterministic systems. To mitigate this problem, Maude provides a strategy language that allows users to define custom execution paths, thereby controlling the rewriting process and restricting the search space instead of exhaustively exploring all possible paths.

Contributions. We present a formal executable semantics for RSs based on a Maude rewrite theory. The proposed theory is fully nondeterministic: at each computation step, the environmental context may be any subset of a fixed pool of external stimuli. This allows the system to explore all potential interactions with the environment without committing in advance to a specific context sequence. To control this high degree of nondeterminism, improve performance, and guide the exploration toward more relevant paths, the theory is equipped with a set of Maude's strategies that acts as a filtering mechanism, pruning the computation tree and retaining only those execution paths that the user considers relevant.

The proposed modeling framework is particularly versatile, as it supports a wide range of analysis tasks beyond simulation. First, strategic rewriting enables reachability analysis by systematically exploring execution paths that satisfy a given set of constraints. Second, the integration with the Unified Maude Model-Checking tool [16] allows for the formal verification of temporal properties expressed as LTL or CTL formulas over strategy-constrained fragments of the computation tree. Third, the nondeterministic context-generation mechanism naturally supports context-sequence synthesis: instead of fixing environmental inputs in advance, the framework can be queried to identify all context sequences that drive the reaction system to states satisfying a given property. Fourth, we allow the expression of both memoryless strategies, which depend only on the current state, and perfect-recall strategies, which may depend on the entire history of previous states. This is particularly useful for testing hypotheses in which the provided context, such as a medical treatment, is generated according to the history of the system's outputs.

To demonstrate the applicability of our approach, we evaluate it on the biological case study presented in [12], which analyses protein signalling networks in HER2-positive breast cancer subtypes under different combinations of monoclonal antibody drugs. Our analyses not only corroborate existing results in the literature but also enable the validation of new hypotheses.

Organization. Section 2 recalls the basics of RSs. Section 3 introduces the necessary background on Rewriting Logic and Maude, while Section 4 presents the Maude executable model for RSs, together with the strategy operators used to control nondeterminism in the system. Section 5 evaluates our approach on the biological case study of [12], showing how strategic rewriting, context-sequence synthesis, memoryless and perfect-recall strategies, and LTL/CTL model checking can be used to reproduce and extend the experimental results of [12]. Finally, Section 6 discusses related work and concludes.

2 Reaction Systems

In this section we recall some basic notions about Reaction Systems (RSs) that are relevant to this work. For a full discussion refer to [6].

We use the term *entities* to denote generic molecular substances (e.g., atoms, ions, molecules, etc.) that may be involved in biochemical reactions. Let S be a finite set of entities. A *reaction* in S is a triple (R, I, P), where R, I, and P are finite sets of entities such that $R, I, P \subseteq S$ and $R \cap I = \emptyset$. The sets R, I, P respectively denote the sets of *reactants*, *inhibitors*, and *products* of the reaction. Due to biological considerations, the sets R and P are usually nonempty: there is no creation from nothing ($R \neq \emptyset$), and if a reaction takes place then something is produced ($P \neq \emptyset$). By $rac(S)$ we denote the set of all reactions over S.

A reaction can take place whenever all of its reactants are present in the current state while none of its inhibitors is present. If this happens, the reaction is *enabled* and creates its products which will be available in the next state. Formally, given a set $W \subseteq S$, and a reaction $a = (R, I, P) \in rac(S)$, the *result* of a on W is defined by

$$res_a(W) \triangleq \begin{cases} P & \text{if } en_a(W) \\ \emptyset & otherwise \end{cases} \qquad en_a(W) \triangleq (R \subseteq W) \wedge (I \cap W = \emptyset)$$

where en_a is the *enabling* predicate.

Let $A \subseteq rac(S)$ be a finite set of reactions. The result of A on W, denoted by $res_A(W)$, is defined by: $res_A(W) \triangleq \bigcup_{a \in A} res_a(W)$. A reaction system is a pair $\mathcal{A} = (S, A)$ such that S is a finite set of entities and $A \subseteq rac(S)$.

RSs are based on three assumptions. First, *no permanency*: any entity vanishes unless it is sustained by a reaction. Second, *no counting*: the basic model of RSs is abstract and qualitative, so the number of occurrences of an entity in a cell is not considered. Third, the *threshold nature of resources*: an entity is assumed to be either available to all reactions or not available at all. As a consequence, nonempty intersections among the sets of reactants of enabled reactions do not generate conflicts. This is a major difference with respect to other models of concurrency, such as Petri nets.

Mimicking living cells, which constantly interact with the environment, a RS is inherently an open system whose behavior is formalized through the notion of *interactive process*, i.e., a process that may react to external stimuli. In our setting, an *environment* is a finite set of entities $Env \subseteq S$. Entities in Env denote

external stimuli that can feed the RS. An interactive process is then formally defined as follows.

Definition 1 (Interactive Process). *Let $\mathcal{A} = (S, A)$ be a RS, $n \geq 0$, and $Env \subseteq S$ be an environment. An n-step* interactive process *in $\mathcal{A}$ w.r.t. Env is a pair $\pi = (\gamma, \delta)$ s.t. $\gamma = \{C_i\}_{i\in[0,n)}$ is the (input)* context sequence *and $\delta = \{D_i\}_{i\in[0,n]}$ is the* result sequence*, where $C_i \subseteq Env$ and $D_i \subseteq S$. Moreover, $D_0 = \emptyset$, and $D_i = res_A(D_{i-1} \cup C_{i-1})$ for any $i \in [1, n]$.*

The context sequence γ represents the environment interacting with the RS and specifies the *input* provided by the environment. The result sequence δ is entirely determined by γ and the reactions in A. In particular, each output D_i depends on the previous output D_{i-1} and the current input C_{i-1}. Thus, distinct context sequences may lead to distinct outcomes for the same reaction set A; however, each context sequence γ determines a *unique* result sequence δ.

Example 1. Let $\mathcal{A} = (S, A)$ be a RS such that $S = \{u, v, w, s_1, s_2\}$, $Env = \{s_1, s_2\}$ and $A = \{a_1, a_2, a_3, a_4\}$ where

$$a_1 = (\{s_1\}, \{s_2\}, \{u, v\}) \qquad a_2 = (\{s_1, s_2, u, v\}, \emptyset, \{w\}).$$
$$a_3 = (\{w\}, \{s_1, s_2\}, \{u\}) \qquad a_4 = (\{u, v, s_1\}, \{s_2\}, \emptyset, \{v\}).$$

Consider a 3-step interactive process $\pi = (\gamma, \delta)$ where $\gamma = C_0, C_1, C_2$ with $C_0 = \{s_1\}$, $C_1 = \{s_1, s_2\}$ and $C_2 = \{s_2\}$. Then $\delta = D_0, D_1, D_2, D_3 = \emptyset, \{u, v\}, \{w\}, \emptyset$.

3 Rewriting Logic and Maude

Maude [7] is a formal language based on rewriting logic (RL) [14] well suited to model concurrent and nondeterministic systems. In RL, an equational theory defines system states as algebraic terms, and rewrite rules define system behavior as local state transitions.

An *equational theory* $\mathcal{E}$ in Maude is defined via a functional module that specifies data types through operators and equations on terms from an order-sorted structure. More precisely, this structure consists of a set of sorts Γ and a poset $(\Gamma, <)$ that models subsort relations. Operators are declared with the syntax `op` f `:` $s_1 \ldots s_n$ `->` s where $s_1 \ldots s_n$ are the sorts of the operator arguments and s is the result sort. Operators with no arguments are called *constants*. Operators can also be declared in mixfix notation using underscores as placeholders (e.g., `op _,_ :` s_1 s_2 `->` s). Binary operators may include algebraic axioms such as associativity, commutativity, and identity; we denote the set of all axioms attached to the operators in the equational theory by `Ax`. Operators are naturally partitioned in two classes: *constructors* (label `ctor` in Maude), which are used to define (irreducible) data values, and *defined* operators, which are used to specify expressions that are evaluated away via equational simplification.

We consider a family of Γ-sorted variables $\mathcal{V}$, where the notation `x:s` denotes a variable `x` of sort `s`. Terms are either variables or applications of operators to lists of terms. *Patterns* are terms that do not contain defined operators. A

substitution $\sigma = \{\mathtt{x_1}/\mathtt{t_1}, \mathtt{x_2}/\mathtt{t_2}, \ldots, \mathtt{x_n}/\mathtt{t_n}\}$ is a (sort-preserving) mapping from the set of variables $\mathcal{V}$ to the set of all terms, which is equal to the identity everywhere except for the set of variables $\{\mathtt{x_1}, \ldots, \mathtt{x_n}\}$. We say that the substitution σ is *ground* if $\mathtt{t_1}, \ldots, \mathtt{t_n}$ do not contain variables.

Equations are specified using the syntax `ceq` l `=` r `if` C, where l and r are terms whose sorts belong to the same connected component of $(\Gamma, <)$ and C is a conjunction of equations that must be satisfied in order for the equation to be applied. When the condition C is empty, we simply write `eq` l `=` r.

Equations are used to simplify terms into their canonical forms modulo `Ax`. This is achieved by orienting the equations from left to right to rewrite (modulo `Ax`) the term to be simplified until no further reductions are possible. To ensure the existence of a unique canonical form for each term, the equational theory is required to be convergent and coherent modulo `Ax` [9].

A *rewrite theory* $\mathcal{R}$ extends the equational theory $\mathcal{E}$ with a set of rewrite rules R, which are specified within a Maude system module. Rewrite rules are defined with the syntax `crl [`lb`] :` l `=>` r `if` C, where lb is a label that identifies the rule, l and r are terms whose sorts belong to the same connected component of $(\Gamma, <)$, and C is a conjunction of equations and rewrite expressions that must be satisfied to enable a rewrite step. When the condition C is empty, we simply write `rl [`lb`] :` l `=>` r. The term l (resp., r) is called *left-hand side* (resp., *right-hand side*) of the rule (similarly for equations).

Rewrite theories formalize concurrent and nondeterministic systems that evolve via rewriting steps modulo the equational theory $\mathcal{E}$. Each transition $t \xrightarrow{lb}_{R,\mathcal{E}} t'$ involves simplifying the term t to its canonical form before applying the rule identified by the label lb. A *computation* in $\mathcal{R}$ corresponds to a (possibly infinite) rewrite sequence $t_0 \xrightarrow{lb_0}_{R,\mathcal{E}} t_1 \xrightarrow{lb_1}_{R,\mathcal{E}} \ldots$, where terms t_i are called *states*. A *computation tree* $T_{\mathcal{R}}(t_0)$ is a tree whose branches represent all computations originating from the initial state t_0. Throughout this paper rewrite theories are also called Maude specifications.

The Maude strategy language [16] is an execution layer built on top of the Maude rewrite engine. It enables users to control rule application by composing combinators—such as sequencing, conditionals, and iteration—thereby transforming a fully nondeterministic rewrite theory into a user-controlled execution model. A strategy is an expression built from the following abstract syntax (we only present the constructors used here).

$\alpha =$ `fail` | `idle` | `lb[`ρ`]` | α ; α | $(\alpha \mid \alpha)$ | `match` P `s.t.` C | α ? α : α |
α`*` | α`+` | `not(`α`)`

where `lb` is a rewrite rule label, ρ is a ground substitution, P is a pattern, and C is an equational condition.

Semantically, a strategy α is a transformation that maps an input state (term) t to the set of all states reachable from t through the computation dictated by α. The strategy `idle` always succeeds and returns the state t unaltered as the only result. The strategy `fail` always fails, thus producing an empty set of resulting states. The strategy `lb[`ρ`]` provides a fine control over rewrite rule applications: it attempts to rewrite the term t by applying the rule with label lb, where the

substitution ρ is used to instantiate some (or all) of the rule's variables before the rule application. The sequential operator α `;` α' models strategy concatenation by first applying α to state t and then applying α' to the states yielded by α. The choice operator α `|` α' executes α or α' on the state t, and the results are both those of α and those of α'. The strategy `match` P `s.t.` C implements a test (without changing the term t) that succeeds when there is a match (modulo the equational theory) between the pattern P and term t with matching substitution θ, and the equational condition $C\theta$ holds. Otherwise this strategy fails. The conditional strategy α_c `?` α `:` α' executes α_c on state t and then α on its results. If α_c does not produce any result (i.e., it fails), this strategy executes α' directly on t. The strategy α`*` (resp., α`+`) specifies the transitive and reflexive closure (resp., the transitive closure) of the strategy α. Finally, the strategy `not(`α`)` is just a convenient notation for the strategy α `? fail : idle`.

Given a strategy α and a term t, the Maude command `srew [`n`]` t `using` α executes α on t and it returns the first n solution. The parameter n is optional.

4 A Maude Specification for Reactions Systems

This section presents our Maude specification that serves as an executable semantics for the RS model introduced in Section 2. This algebraic specification provides a formal and computable implementation for interactive processes. More precisely, given a RS $\mathcal{A} = (S, A)$, we define a rewrite theory $\mathcal{R}_\mathcal{A}$ that includes: (i) an equational theory $\mathcal{E}_\mathcal{A}$ that implements the basic data structures and set-theoretic operations which are required to model RS states; (ii) a set of rewrite rules $R_\mathcal{A}$ acting on top of $\mathcal{E}_\mathcal{A}$ specifying the system behavior of $\mathcal{A}$. We say that $\mathcal{R}_\mathcal{A}$ is the *the Maude encoding* of $\mathcal{A}$. The core type structure along with the most relevant operators of $\mathcal{E}_\mathcal{A}$ and the rule set $R_\mathcal{A}$ are shown in Figure 1. We omit standard auxiliary definitions (e.g., equational definitions for the set operators `subset`, `isDisjoint`, etc.). The complete Maude source code is available at [3].

Entities in $\mathcal{E}_\mathcal{A}$ are modeled as constants of sort `Entity`. Sets of entities (sort `Entities`) are built from the associative, commutative with identity `empty` juxtaposition operator `op __` (Lines 5–6). Then, given two entities `e` and `egf`, the term `e egf`, of sort `Entities`, represent the set $\{e, egf\}$. Idempotency equation in Line 7 enforces the standard set semantics by collapsing duplicate entities.

The sort `Sequence` along with the associative operator `op _,_` model a comma-separated list of entity sets, which will be used to represent both the context sequence supplied by the environment and the result sequence accumulated during computation. The identity `nil` specifies the empty sequence (Lines 9–10). Since the sort `Entity` is a subsort of `Entities` which in turn is a subsort of `Sequence` (Line 2), any entity or entity set is also a term of sort `Sequence`. For instance, the term `egf e , e` is a well-typed sequence provided that `egf` and `e` are terms of sort `Entity`. Reactions are terms of sort `Reaction` of the form `[R,I,P]`, where `R`, `I`, and `P` are sets of entities that respectively specify reactants, inhibitors and products of the reaction (Line 11).

```
1  sorts Entity Entities Sequence Reaction Reactions .
2  subsort Entity < Entities < Sequence .
3  subsort Reaction < Reactions .
4
5  op empty : -> Entities [ctor] .
6  op __ : Entities Entities -> Entities [ctor assoc comm id: empty] .
7  eq X X = X . --- Idempotency (X of sort Entity)
8
9  op nil : -> Sequence [ctor] .
10 op _,_ : Sequence Sequence -> Sequence [ctor assoc id:nil] .
11 op [_,_,_] : Entities Entities Entities -> Reaction [ctor] .
12
13 op empty : -> Reactions [ctor] . .
14 op _;_ : Reactions Reactions -> Reactions [ctor assoc id: empty] .
15
16 op en : Reaction Entities -> Bool .
17 eq en([R,I,P],T) = (R subset T) and-then isDisjoint(I,T) .
18
19 op res : Reaction Entities -> Entities .
20 eq res([R,I,P],T) = if en([R,I,P],T) then P else empty fi .
21 op res : Reactions Entities -> Entities .
22 eq res(empty,T) = empty .
23 eq res((r ; Rs),T) = res(r,T) res(Rs,T) .
24
25 op <_|_|_> : Reactions Sequence Sequence -> State [ctor] .
26 crl [next] :  < A | Ds , D | Cs > =>
27              < A | Ds , D , res(A, C D) | Cs , C > if C C' := Env .
```

Fig. 1. The rewrite theory $\mathcal{R}_{\mathcal{A}}$.

The sort `Reactions` and its associated operator `op _;_` model an ordered, semicolon-separated list of reactions representing the full reaction set A of a RS. The constant `empty` is again used to specify an empty reaction list (Lines 13–14). Due to the subsort relation `Reaction < Reactions` (Line 3), any individual `Reaction` term is also a valid `Reactions` term.

The function `res([ R,I,P ], T)` (Lines 19–20) computes the output of a single reaction (R, I, P) fired against a given entity set T (i.e., $res_{(R,I,P)}(T)$), delegating the firing condition check to the predicate `en([ R,I,P ],T)` (Lines 16–17). Note that `en([ R,I,P ],T)` is a direct and straightforward implementation of the enabling predicate defined in Section 2. Consequently, the function `res` returns the term `P` (i.e., the products) if `en([ R,I,P ],T)` evaluates to true, and `empty` otherwise. The function `res(A,T)` (Lines 22–23) computes the set of entities $res_A(T)$ by recursively applying all the reactions in the reaction list `A` to `T`.

System states (or simply states) are terms of the form `< A | Ds | Cs >`, of sort `State`, where `A` is a term specifying the list of reactions of the RS, `Ds` is the

result sequence accumulated so far, and `Cs` is the context sequence supplied by the environment (line 25).

For a fixed set of external stimuli *Env*, the behavior of a RS $\mathcal{A} = (S, A)$ is completely captured by the single rewrite rule `next` (Lines 26–27) that combines two phases: context generation and reaction activation. Context generation is achieved by using the rule condition `C C' := Env` that nondeterministically (due to the commutativity and associativity of `op __`) splits `Env` into a context `C` and a remainder `C'`. Hence, every possible subset `C` of `Env` can be generated during the application of this rule. Reaction activation is then performed by computing the new result set `res(A,C D)`, where `C` is the freshly generated context and `D` is the most recently accumulated result. Finally, the new result set `res(A,C D)` and context `C` are appended to their respective sequences, updating the current state. Therefore, computations in $\mathcal{R}_\mathcal{A}$ take the form:

$$\langle \mathtt{A} \,|\, \mathtt{D}_0 \,|\, \mathtt{nil} \rangle \xrightarrow{\mathtt{next}}_{R_\mathcal{A},\mathcal{E}_\mathcal{A}} \langle \mathtt{A} \,|\, \mathtt{D}_0, \mathtt{D}_1 \,|\, \mathtt{C}_0 \rangle \xrightarrow{\mathtt{next}}_{R_\mathcal{A},\mathcal{E}_\mathcal{A}} \langle \mathtt{A} \,|\, \mathtt{D}_0, \mathtt{D}_1, \mathtt{D}_2 \,|\, \mathtt{C}_0, \mathtt{C}_1 \rangle \xrightarrow{\mathtt{next}}_{R_\mathcal{A},\mathcal{E}_\mathcal{A}} \ldots$$

where, at each rewrite step, the `next` rule uses the result set $\mathtt{D_i}$ and a freshly generated context $\mathtt{C_i}$ to compute the next result $\mathtt{D_{i+1}}$, precisely specifiying an n-step interactive process w.r.t. a given environment. Furthermore, computational steps are preserved in both directions: each transition in an interactive process in A corresponds to exactly one rewrite step in $\mathcal{R}_\mathcal{A}$, and vice-versa, which provides an operational correspondence between the two computational models.

Theorem 1 (Operational correspondence). *Let $\mathcal{A} = (S, A)$ be a RS, $Env \subseteq S$, and $\mathcal{R}_\mathcal{A}$ be the Maude encoding of $\mathcal{A}$. Let also $\pi = (\gamma, \delta)$ s.t $\gamma = \{C_i\}_{i\in[0,n)}$, $\delta = \{D_i\}_{i\in[0,n]}$, $n \geq 0$, $C_i \subseteq Env$ and $D_i \subseteq S$. Then, π is an n-step interactive process in $\mathcal{A}$ iff $\langle \mathtt{A} \,|\, \mathtt{D}_0 \,|\, \mathtt{nil} \rangle \xrightarrow{\mathtt{next}}_{R_\mathcal{A},\mathcal{E}_\mathcal{A}} \ldots \xrightarrow{\mathtt{next}}_{R_\mathcal{A},\mathcal{E}_\mathcal{A}} \langle \mathtt{A} \,|\, \mathtt{D}_0, \ldots, \mathtt{D_n} \,|\, \mathtt{C}_0, \ldots \mathtt{C}_{n-1} \rangle$ where $\mathtt{C}_0, \mathtt{C}_1, \ldots, \mathtt{C}_{n-1}$ and $\mathtt{D}_0, \mathtt{D}_1, \ldots, \mathtt{D_n}$ are the terms respectively encoding the sequences γ and δ of π.*

Example 2. Let $\mathcal{A}$=(A,S) be the RS of Example 1 where $Env = \{s_1, s_2\}$. Let $\mathcal{R}_\mathcal{A}$ be the Maude encoding of $\mathcal{A}$. Then, there exists a computation in $\mathcal{R}_\mathcal{A}$:

$$\begin{aligned} < \mathtt{A} \,|\, \mathtt{empty} \,|\, \mathtt{nil} > &\xrightarrow{\mathtt{next}}_{R_\mathcal{A},\mathcal{E}_\mathcal{A}} < \mathtt{A} \,|\, \mathtt{empty}, \mathtt{u\ v} \,|\, \mathtt{s_1} > \\ &\xrightarrow{\mathtt{next}}_{R_\mathcal{A},\mathcal{E}_\mathcal{A}} < \mathtt{A} \,|\, \mathtt{empty}, \mathtt{u\ v}, \mathtt{w} \,|\, \mathtt{s_1}, \mathtt{s_1\ s_2} > \\ &\xrightarrow{\mathtt{next}}_{R_\mathcal{A},\mathcal{E}_\mathcal{A}} < \mathtt{A} \,|\, \mathtt{empty}, \mathtt{u\ v}, \mathtt{w}, \mathtt{empty} \,|\, \mathtt{s_1}, \mathtt{s_1\ s_2}, \mathtt{s_2} > \end{aligned}$$

where `empty,u v,w, empty` and `s_1,s_1 s_2,s_2` respectively specify the result and context sequences of the 3-step interactive process π from Example 1.

Example 2 illustrates a specific three-step interactive process in the RS $\mathcal{A}$ that can be computed by $\mathcal{R}_\mathcal{A}$ with respect to the environment $Env = \{s_1, s_2\}$. However, because of the nondeterministic nature of $\mathcal{R}_\mathcal{A}$, this theory computes all n-step interactive processes with respect to Env. Indeed, each application of the `next` rule nondeterministically selects an arbitrary subset of Env as the context, thereby generating $2^{|Env|}$ possible branches at each rewrite step. This leads to a combinatorial explosion that can quickly become intractable. Below, we address

```
1  sd se(0,Req) := idle .
2  sd se(s(N),Req)) := next [ C <- Req ] ; se(N, Req) .
3
4  sd se-sets(N,Req) := se(N, Req) .
5  sd se-sets(N,(Req,S')) := se(N, Req) | se-sets(N,S') .
6
7  sd se-cond (Ctx,Req,Excl,N) := match < R | Ds | Cs >
8                                    s.t. suffixCheck(Ds,Req,Excl,N) = true ;
9                                 next [ C <- Ctx ] .
10
11 sd s(0,Req,Excl) := idle .
12 sd s(s(N),Req,Excl) := next ; match < R | Ds | Cs , (Req C) >
13                                 s.t. isDisjoint(Excl,(Req C)) = true ;
14                         s(N, Req, Excl) .
15
16 sd s-all(N) := s(N,empty,empty) .
17 sd s-atleast(N,Req) := s(N,Req,empty) .
18 sd s-without(N,Excl:E) := s(N,empty,Excl) .
19
20 sd isSteady := match < R | Ds , D , D | Cs  > .
21 sd se-steady(Req) := isSteady ? idle : (se(1,Req) ; se-steady(Req)) .
22 sd s-steady(Req,Excl) := isSteady ? idle : s(1,Req,Excl) ;
23                                           s-steady(Req,Excl) .
24 sd in(Req) := match < R | Ds , (Req D) | Cs > .
```

Fig. 2. Strategy language for RSs

this problem by formalizing suitable rewriting strategies in the Maude strategy language. These strategies control nondeterminism and guide the search toward relevant computations only.

Rewrite Strategies for $\mathcal{R}_\mathcal{A}$. Maude's operator `sd` allows for the definition of named parameterized strategies that can be recursive and can freely invoke any (auxiliary) equationally defined function. This expressiveness allows us to design the strategy language in Figure 2 specifically tailored to exploring the iterative processes generated by $\mathcal{R}_\mathcal{A}$.

The recursive strategy `se(N,Req)` (Lines 1–2) takes as input a natural number `N` and a set of entities `Req` and generates an `N`-step computation in $\mathcal{R}_\mathcal{A}$, where —at each rewrite step— the context injected is exactly `Req`, hence producing a totally deterministic N-step interactive process. This is achieved by recursively applying the `next` rule under the substitution `[C <- Req]`, which enforces the required context at each rewrite. The strategy `se-sets(N,S)` (Lines 4–5) is a nondeterministic generalization of `se(N,Req)` that uses the choice operator (`|`) to select, in parallel, every set of entities from the sequence `S` as the fixed context, yielding the corresponding `N`-step computation.

Contexts can also be generated conditionally via `se-cond(Ctx,Req,Excl,N)` (Lines 7–9), which injects context `Ctx` in the next step whenever the last `N` result

sets of the current result sequence include the set of entities `Req` and exclude `Excl`. The condition is checked by the auxiliary function `suffixCheck(Ds,Req,Excl,N)` (see [3] for its definition) which returns true exactly when this holds. Note that this strategy allows the system to evolve based on results computed in the past, making context injection responsive to the history of the computation.

The recursive strategy `s(N,Req,Excl)` (Lines 11–14) nondeterministically explores all possible `N`-step computations whose context, at each step, includes all entities in `Req` and excludes all entities in `Excl`. Unlike the `se` strategy, entities in the environment `Env` beyond `Req` are allowed, as long as they are not in `Excl`. This is implemented via a `match` test that (i) checks the inclusion of `Req` by matching the last element of the result sequence against the decomposition `(Req C)`, and (ii) checks the exclusion of `Excl` by evaluating `isDisjoint` on `Excl` and `(Req C)`. The strategies `se-all`, `se-atleast`, and `se-without` are all convenient specializations of `s(N,Req,Excl)` (Lines 16–18) obtained by instantiating `Req` and/or `Excl` to the empty set. In particular, `se-all(N)` performs an unrestricted `N`-step exploration that produces all N-step interactive processes w.r.t. Env.

The strategy `isSteady` (Line 20) tests whether the computation has reached a steady state, i.e., whether the last two sets in the current result sequence are identical. The strategy `in(Req)` (Line 24) checks whether all entities in `Req` are included in the most recently computed result set.

Finally, the strategies `se-steady(Req)` (Line 21) and `s-steady(Req,Excl)` (Lines 22–23) both rely on the conditional operator to iterate until a steady state is reached: at each recursive call, `isSteady` is tested and, if it holds, the strategy halts via `idle`; otherwise, one further recursive step is taken.

In the next section we show how this strategy language can be effectively used to formally analyze a biological system.

5 Analyzing a Biological Case Study

We consider the case study from [12] concerning protein signaling networks in breast cancer and the associated drug therapies used to inhibit the metabolic pathways responsible for tumor genesis and progression. The *in silico* experimental analyses in [12] focus on therapies where different drug combinations are tested against distinct breast cancer subtypes considering: (i) three kinds of HER2-positive subtype breast cancer, identified by the cell lines BT474, HCC1954, and SKBR3; (ii) three types of drugs: erlotinib (e), pertuzumab (p), trastuzumab (t), provided alone or in combination; (iii) two types of input stimuli, EGF and HRG, which start the signaling pathways to be analyzed; (iv) two different treatment periods: short- and long-term drug treatments.

The experiments in [12] searches for the presence of key molecules, which are responsible for tumors, in *attractor* states (in short, attractors). Attractor represent those states toward which the system tends to evolve over time. Specifically, the short-term analysis searches for molecules AKT, ERK1/2, and p70S6K in attractors, while the long-term analysis searches for RPS6 and RB.

	Short-term									Long-term					
	BT474			HCC1954			SKBR3			BT474		HCC1954		SKBR3	
	AKT	ERK1/2	p70S6K	AKT	ERK1/2	p70S6K	AKT	ERK1/2	p70S6K	RPS6	RB	RPS6	RB	RPS6	RB
x	1	1	1	1	1	1	1	1	1	1	0	1	0	1	0
e	0	1	1	1	0	1	0	0	0	0	0	0	0	0	0
p	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0
t	1	1	1	1	1	1	1	1	1	1	0	1	0	1	0
e,p	1	1	1	1	0	1	0	0	0	0	0	0	0	0	0
e,t	0	1	1	1	0	1	0	0	0	0	0	0	0	0	0
p,t	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0
e,p,t	1	1	1	1	0	1	0	0	0	0	0	0	0	0	0

Table 1. Short-term and long-term experiments from [12].

Table 1 summarizes the results of [12] for the six considered systems: one per kind of cancer cell line, short- and long-term versions. In the first column the drug combinations are reported (x stands for no drug treatment). Each row of the table presents the efficacy of a drug combination under the same input stimulus for the three kinds of cancer. The treatment is effective when the molecules of interest (AKT, ERK1/2, p70S6K, for short-term, and RPS6, RB for long-term) are not present in attractor states, i.e. the cell value in the table is equal to 0.

To reproduce the experiments in Table 1 in our verification framework, we reused the six RSs from our previous work [2], which model the reactions for the short-term and long-term analyses of the considered cancer cell lines, respectively. These RSs were directly derived from the Boolean signaling networks of [12] using the translation method proposed in [5]. The entities within these RSs specify every individual component of the system (proteins, enzymes, etc.), while the reactions define their dynamic behavior. For instance, the following code snippet shows a fragment of the constant operator `BT474`, which specifies the reactions used in the short-term experiments for the cancer cell line BT474:

```
op BT474 : -> Reactions  .
eq BT474 =  [ akt , empty , akt ] ; [ egf , e p , erbb1 ] ; ...
```

The resulting RSs for the six systems listed in Table 1 (from left to right) consist of 39, 39, 34, 47, 50, and 55 reactions, respectively. Their full specifications, along with the data necessary for readers to fully reproduce the experiments are in [3]. The environment from which contexts are generated includes the input stimuli EGF and HRG and the three drugs e, p, and t. This environment is modeled as a set of entities by the constant (of sort `Entities`) `Env = egf hrg e p t`.

Notably, all the experiments can be modeled as reachability properties that check for the presence of a specific molecule in an interactive process. For instance, the strategy `s(1,e t,p) ; se-steady(e t) ; includes(akt)` verifies the efficacy of the drug combination `e t` in the BT474 cancer subtype. This strategy executes an initial step using a context that must include `e` and `t` but excludes the drug `p`. This initial context may also contain any combination of the input stimuli `egf` and `hrg` (present in the constant `Env`) required to activate the metabolic process. Subsequent steps are then executed using a context containing only `e` and `t` until a steady state (i.e., an attractor) is reached. Finally, the target molecule `akt` is checked within the attractor state. We can therefore run the strategic search from the initial state ⟨`BT474 | empty | nil`⟩:

```
srew < BT474 | empty | nil > using s(1,e t,p) ; se-steady(e t) ; in(akt) .
No solution.
```

The output of this command demonstrates the effectiveness of treatment that combines `e` and `t`. On the contrary, the following strategic search for the combined drugs `e` and `p` produces four solutions. Each of these serves as a valid counterexample highlighting the limitations of this drug combination.

```
srew < BT474 | empty | nil > using s(1,e p,t) ; se-steady(e p) ; in(akt) .
[...]
Solution 4
result State: < ... | empty, mtor plcg erk12, akt p70s6k mtor plcg erk12
                      pkca, akt p70s6k mtor plcg erk12 pkca
                    | e p egf hrg, e p, e p >
No more solutions.
```

Using similar strategies, we successfully reproduced all the results presented in Table 1. The execution times on a MacBook Pro M4 with 32 GB of RAM were negligible, requiring only a few milliseconds per experiment.

Beyond reproducing the results in [12], our framework is expressive enough to test new hypotheses and evaluate more complex scenarios as detailed below.

Context Sequence Synthesis. Our framework enables the automatic synthesis of context sequences that satisfy a given property. This capability is particularly valuable when searching for potential drug treatments that are successful against a specific key molecule. Consider, for instance, a bounded strategic search over the computation tree of the HCC1954 cancer subtype. The objective is to identify all potential treatments that, after exactly 6 sessions, reach an attractor state where `akt` is absent. For this search, we require that the contexts consistently include the input stimulus `egf` and the drug `e`, while strictly excluding the input stimulus `hrg` and the drug `p`.

```
srew  < HCC1954 | empty | nil > using s(6,egf e,hrg p) ; isSteady ;
                                     not(in(erk12)) .
[...]
Solution 63
result State: < ...| empty, p70s6k pkca, akt p70s6k pkca, akt p70s6k  ...
                    e t egf, e t egf, e t egf, e t egf >
No more solutions.
```

The search yields 63 solutions, where each context sequence in the final state represents a treatment satisfying the specified property. In the solution above, the treatment consists in using a combination of drugs `e` and `t`.

Instead of limiting the depth of the search space as we did above, we can just limit the number of solutions to be delivered:

```
srew [10] < HCC1954 | empty | nil > using s(1,egf e,hrg p) +  ; isSteady ;
                                       not(in(erk12)) .
```

This command returns the first 10 solutions that meet the previously specified stimulus and drug constraints, while removing the depth restriction on the computation tree (i.e., the 6-session treatment limit).

Perfect Recall Context Strategies. Thanks to the `se-cond` operator, we can generate conditional context sequences that depend on the result sequence computed so far. This makes it possible to implement perfect-recall strategies, in which new contexts are generated according to the presence or absence of specific entities in the previous outputs. As an example, consider the `BT474` cancer subtype under the persistent input stimulus `egf`. Suppose that, during the first five treatment sessions, we administer the drugs `e` and `t` to inhibit the `akt` molecule. We then want to determine whether it is possible to drop `t` from the treatment and still reach an attractor that does not contain `akt`, provided that `akt` is absent from the last three computed result sets. This reachability problem can be specified by means of the following strategic search:

```
srew [1] < BT474 | empty | nil > using se(5,egf e t) ;
                 se-cond(egf e,empty,akt,3)+ ; isSteady ; not (in(akt)) .
```

This strategy executes exactly five steps under the fixed context `egf e t`. Then, `se-cond(...)+` repeatedly fires single steps with the context `egf e`, but only as long as the three most recent result sets exclude `akt`. Finally, we check that the system has reached an attractor and it does not contain `akt`. This command returns a solution confirming that we can safely withdraw drug `t` after five steps and still reach a stable configuration without `akt`.

LTL and CTL Model-checking. The `umaudemc` tool [16] is an interface to various model checkers for Maude models. Below we demonstrate how we can verify LTL and CTL formulas against our models for HER2-positive cancers. Temporal formulas are built using atomic formulas, Boolean connectives, and standard temporal operators. For LTL we have $\Box$ (always), $\Diamond$ (eventually), $\circ$ (next), *etc.* For CTL we further have the path quantifiers **A**ll and **E**xists.

Temporal logic formulas can be model-checked via the `check` command of `umaudemc`, which executes the verification by taking as input the Maude specification file, the initial state defined as a term of sort `State`, the target temporal formula, and an optional Maude strategy parameter that allows properties to be model-checked exclusively on the fragment of the computation tree identified by that strategy, thereby reducing the search space.

More concretely, the results of [12], reproduced above with the `srew` command can alternatively be obtained via model-checking. For instance,

```
umaudemc check SKBR3.maude  "< SKBR3 | empty | nil >"
                 "<> ( p70s6k ∧ steady )"  "s(1,e p,t) ; se-steady(e p)"
```

checks for the presence of `p70s6k` in attractors of the SKBR3 cancer subtype when drugs `e` and `p` are continuously administered. This is achieved by verifying the LTL formula $\Diamond$`(p70s6k ∧ steady)`, where `p70s6k` and `steady` are atomic formulas respectively specifying the presence of `p70s6k` and whether the current state is an attractor. Note that the verification is performed only on a fragment of the computation tree for the initial state ⟨`SKBR3 | empty | nil`⟩, identified by the strategy `s(...) ; se-steady(e p)`. This strategy first performs a single step with a context that includes drugs `e` and `p` (but not `t`) plus any possible combination

of input stimuli `egf` and `hrg`, then repeatedly applies the fixed context `e p` until the system reaches a steady state.

CTL formulas allow us to assess the efficacy of distinct treatments in parallel. Consider, for instance, three 10-step treatments for the `BT474` cancer subtype, where the drugs `e`, `p`, and `t` are administered, respectively, under the persistent input stimulus `egf`. To ask whether at least one treatment admits a run that reaches an attractor in which `akt` is absent, we verify the CTL formula $\mathbf{E}\Box(\texttt{steady} \rightarrow \neg\texttt{akt})$ using the following command:

```
umaudemc check BT474.maude  "⟨BT474 | empty | nil⟩"
            "E [] ( steady -> ~ akt )"  "se-sets(10,(egf e,egf p,egf t))"
```

The formula is satisfied, consistently with Table 1, which shows that `e` inhibits `akt` production in `BT474`. The universally quantified formula $\mathbf{A}\Box(\texttt{steady} \rightarrow \neg\texttt{akt})$, instead, does not hold since drugs `p` and `t` are not effective against `akt`.

6 Conclusions and Related Work

We presented a formal executable semantics for RSs implemented as a fully nondeterministic rewrite theory, where Maude strategies control the combinatorial explosion due to unrestricted context generation. The applicability of this approach has been demonstrated through complex analysis tasks including strategic reachability analysis, context sequence synthesis, perfect recall strategies, and LTL/CTL model checking. Our results have been validated against a real-size biological case study on drug treatments in HER2-positive breast cancer.

There are several analysis and verification frameworks for RSs that can be compared to ours. [1] provides a Maude formalization of RSs which is suitable for trace slicing, a technique used to simplify computations and facilitate bug detection. In [1] context sequences are manually given, whereas our approach nondeterministically yields all possible context sequences, automatically.

In [2] we introduced the process algebra ccReact, which is well suited to RS simulation and verification. Like the strategy-based approach presented here, ccReact extends the classical RS model by supporting recursive, nondeterministic, and conditional context sequences. However, conditional context generation in ccReact depends strictly on the entities in the current system state. In contrast, the framework proposed here can naturally define "perfect-recall" strategies that depend on the entire computation. Furthermore, context-sequence synthesis is not supported in ccReact. We also note that users of [2] must design process-algebra expressions to define contexts. This task is simplified here thanks to an intuitive strategy language. Finally, we provide here a better control of nondeterminism through Maude's strategy-language implementation, which supports multithreading and depth-first search analyses.

The authors of [13] define a CTL logic for context-restricted RSs, while [15] introduces an LTL logic for a multiset-based version of RSs with discrete concentrations. A key difference is that both [13,15] rely on custom model checkers. In contrast, our framework utilizes standard Maude model-checkers and avoids the need of developing/maintaining *ad-hoc* verification tools.

References

1. Ballis, D., Brodo, L., Falaschi, M.: Modeling and analyzing reaction systems in maude. Electronics **13**(6,1139) (2024). https://doi.org/https://doi.org/10.3390/electronics13061139
2. Ballis, D., Brodo, L., Falaschi, M., Olarte, C.: ccReact: a rewriting framework for the formal analysis of reaction systems. Int. J. Softw. Tools Technol. Transf. **26**(6), 707–725 (2024). https://doi.org/10.1007/S10009-024-00772-Z
3. Ballis, D., Brodo, L., Falaschi, M., Olarte, C.: Smarts: Strategy-based analyzer for reaction systems. Available at https://github.com/DemisGIT/SMARtStool (2026)
4. Barbuti, R., Gori, R., Levi, F., Milazzo, P.: Investigating dynamic causalities in reaction systems. Theor. Comput. Sci. **623**, 114–145 (2016). https://doi.org/https://doi.org/10.1016/j.tcs.2015.11.041
5. Barbuti, R., Gori, R., Milazzo, P.: Encoding boolean networks into reaction systems for investigating causal dependencies in gene regulation. Theor. Comput. Sci. **881**, 3–24 (2021). https://doi.org/https://doi.org/10.1016/j.tcs.2020.07.031
6. Brijder, R., Ehrenfeucht, A., Main, M., Rozenberg, G.: A tour of reaction systems. Int. J. Found. Comput. Sci. **22**(07), 1499–1517 (2011). https://doi.org/https://doi.org/10.1142/S0129054111008842
7. Clavel, M., Durán, F., Eker, S., Escobar, S., Lincoln, P., Martí-Oliet, N., Meseguer, J., Rubio, R., Talcott, C.: Maude Manual (Version 3.5.1). Tech. rep. (2025), https://maude.lcc.uma.es/maude-manual/
8. Corolli, L., Maj, C., Marinia, F., Besozzi, D., Mauri, G.: An excursion in reaction systems: From computer science to biology. Theor. Comput. Sci. **454**, 95–108 (2012). https://doi.org/https://doi.org/10.1016/j.tcs.2012.04.003
9. Durán, F., Eker, S., Escobar, S., Martí-Oliet, N., Meseguer, J., Rubio, R., Talcott, C.L.: Programming and symbolic computation in maude. J. Log. Algebraic Methods Program. **110** (2020). https://doi.org/10.1016/j.jlamp.2019.100497
10. Ehrenfeucht, A., Main, M.G., Rozenberg, G.: Combinatorics of life and death for reaction systems. Int. J. Found. Comput. Sci. **21**(3), 345–356 (2010). https://doi.org/10.1142/S0129054110007295
11. Ehrenfeucht, A., Main, M.G., Rozenberg, G.: Functions defined by reaction systems. Int. J. Found. Comput. Sci. **22**(1), 167–178 (2011). https://doi.org/10.1142/S0129054111007927
12. der Heyde, S.V., Bender, C., Henjes, F., Sonntag, J., Korf, U., Beißbarth, T.: Boolean ErbB network reconstructions and perturbation simulations reveal individual drug response in different breast cancer cell lines. BMC Systems Biology **8**(1), 75 (2014). https://doi.org/10.1186/1752-0509-8-75
13. Męski, A., Penczek, W., Rozenberg, G.: Model checking temporal properties of reaction systems. Information Sciences **313**, 22–42 (2015). https://doi.org/10.1016/j.ins.2015.03.048
14. Meseguer, J.: Twenty years of rewriting logic. J. Log. Algebraic Methods Program. **81**(7-8), 721–781 (2012). https://doi.org/10.1016/J.JLAP.2012.06.003
15. Meski, A., Koutny, M., Penczek, W.: Verification of linear-time temporal properties for reaction systems with discrete concentrations. Fundam. Informaticae **154**(1-4), 289–306 (2017). https://doi.org/10.3233/FI-2017-1567
16. Rubio, R., Martí-Oliet, N., Pita, I., Verdejo, A.: Strategies, model checking and branching-time properties in maude. J. Log. Algebraic Methods Program. **123**, 100700 (2021). https://doi.org/10.1016/J.JLAMP.2021.100700